

ORDER FLOW SOFTWARE

External Developer SDK & Indicator Exposure Specification

*How external developers consume, compose, and build trading tools
on top of the Order Flow Engine's live indicator outputs*

v1.0 | May 2026 | Confidential & Proprietary

1. Overview — The Developer Platform Vision

The Order Flow Engine exposes every indicator, signal, and computed metric it generates as a structured, authenticated, real-time data platform that external developers can consume to build their own trading scripts, indicators, automated strategies, and tools.

This is not just a data feed. It is a full developer SDK with three layers:

Layer	What It Provides	Who Uses It
Layer 1 — Streaming API	Real-time WebSocket stream of all live indicator outputs: delta, CVD, imbalances, VWAP, Volume Profile, Pulse, Turns, Ratio signals	Algorithmic traders, automated strategy developers, real-time dashboards
Layer 2 — Historical REST API	Query historical computed indicator values for backtesting, strategy validation, and statistical research	Quantitative analysts, backtesters, strategy developers
Layer 3 — Native Plugin SDK	C++ / C# / Python SDK allowing developers to write custom indicators that plug directly into the engine's data pipeline	Advanced developers building custom indicators on top of OFE data

Every indicator described in the main specification — Footprint, Delta, CVD, Imbalances, Volume Profile, VWAP, Pulse, Turns, Ratio, Single Prints, Delta Surge, and all 30+ Trader 7 signals — is exposed as a named, versioned, subscribable data stream with a well-defined JSON schema.

***i** External developers never access the core C++ engine directly. They access computed outputs only — through authenticated, rate-limited API endpoints. The engine's IP is fully protected.*

2. SDK Architecture

2.1 Three Access Tiers

Tier	Access Method	Data Available	Subscription Level	Rate Limits
Basic	REST API only	Historical bars, EOD Volume Profile, daily VWAP	Monthly Basic	100 req/min, no WebSocket
Standard	REST API + WebSocket streaming	All historical + live delta, CVD, imbalances, Volume Profile, VWAP, VWAP bands	Monthly Standard	1,000 req/min, 5 WebSocket connections
Professional	REST + WebSocket + Plugin SDK	Everything in Standard + Pulse, Turns, Ratio, Single Prints, all Trader 7 signals	Monthly Professional	10,000 req/min, 25 WebSocket connections, plugin SDK access
Enterprise	Full access + SLA + custom endpoints	All data + priority queue + custom signal definitions + raw tick stream	Annual Enterprise	Unlimited, 100 WebSocket, dedicated infrastructure, white-label rights

2.2 Authentication Flow

Every developer registers at the developer portal and receives credentials:

Step 1 — Developer Registration

- Developer registers at portal.orderflow-engine.com/developers
- Selects subscription tier and billing cycle
- Creates an application (one API key set per application)

Step 2 — API Key Issuance

- Public Key: `ofe_pk_live_XXXXX` (used in all REST requests)
- Secret Key: `ofe_sk_live_XXXXX` (used to sign WebSocket connections — never exposed client-side)
- Keys are scoped to specific indicator groups: e.g., `scope=delta,vwap,imbalances`
- Keys can be restricted by IP address range for server-side applications

Step 3 — JWT Token Exchange (for WebSocket)

WebSocket connections use short-lived JWT tokens (15 min) obtained by exchanging the API key pair:

```
POST /v1/auth/token
Authorization: Basic base64(ofe_pk_live:ofe_sk_live)
Response: { access_token, expires_in: 900, token_type: Bearer }
```

Step 4 — Ongoing Access

- REST: include ofe_pk_live in header X-OFE-API-Key
- WebSocket: connect with ?token=<jwt_access_token>
- Tokens refresh automatically if developer uses the SDK client libraries

2.3 SDK Client Libraries

Official client libraries provided and maintained to make integration fast:

Language	Library	Installation	Minimum Runtime
Python	ofe-python-sdk	pip install ofe-sdk	Python 3.9+
C++	ofe-cpp-sdk	CMake FetchContent or vcpkg	C++17, Boost.Asio
C#	OFE.SDK.DotNet	NuGet: Install-Package OFE.SDK	NuGet .NET 6+
JavaScript / Node.js	ofe-js-sdk	npm install @ofe/sdk	Node 18+
Rust	ofe-rs	cargo add ofe-sdk	Rust 1.70+
Pine Script Bridge	ofe-pine-bridge	Import via TradingView script	TradingView Pro+

***i** All SDK libraries handle authentication, reconnection, heartbeats, and deserialization automatically. Developers focus on signal logic, not infrastructure.*

3. Complete Indicator Exposure Catalog

Every indicator produced by the engine is exposed with a REST endpoint for historical data and a WebSocket topic for real-time streaming. Schemas are versioned (v1, v2...) and backwards-compatible within major versions.

3.1 Footprint Chart Data

REST Endpoint

```
GET /v1/footprint/{instrument}/{bar_type}/{bar_size}
Parameters: from_ts, to_ts, include_price_levels=true
```

WebSocket Topic

```
SUBSCRIBE footprint:{instrument}:{bar_type}:{bar_size}
```

Payload Schema — Bar Record

Field	Type	Description
bar_id	string	Unique bar identifier: {instrument}_{timestamp}_{bar_type}
open / high / low / close	float64	Standard OHLC prices
bar_open_ts / bar_close_ts	int64	Nanosecond timestamps for bar open and close
total_volume	int64	Total contracts traded in bar
bar_delta	int32	ask_vol - bid_vol for entire bar
max_delta	int32	Maximum cumulative delta reached within bar
min_delta	int32	Minimum cumulative delta reached within bar
cumulative_delta	int64	Session-running CVD up to and including this bar
poc_price	float64	Price level with highest volume in bar (bar POC)
cot_price	float64	Commitment of Traders: price with highest combined bid+ask
is_bullish	bool	true if close > open
price_levels	array	Array of PriceLevelRecord objects (see below)

Payload Schema — Price Level Record

Field	Type	Description
price	float64	This price level
bid_vol	int32	Volume executed on bid at this price (aggressive sellers / passive buyers)

ask_vol	int32	Volume executed on ask at this price (aggressive buyers / passive sellers)
total_vol	int32	bid_vol + ask_vol
delta	int32	ask_vol - bid_vol at this level
is_buy_imbalance	bool	ask_vol / bid_vol_diagonal >= imbalance_ratio
is_sell_imbalance	bool	bid_vol / ask_vol_diagonal >= imbalance_ratio
is_poc	bool	This is the bar's Point of Control
is_cot	bool	This is the bar's Commitment of Traders level

3.2 Delta & Cumulative Delta

REST — Historical Delta

```
GET /v1/delta/{instrument}/{bar_type}/{bar_size}
Parameters: from_ts, to_ts, metrics=bar_delta,max_delta,min_delta,cvd,delta_pct
```

WebSocket Topics

```
SUBSCRIBE delta:live:{instrument} // bar-building delta, updates every 100ms
SUBSCRIBE delta:bar:{instrument} // final bar delta on bar close
SUBSCRIBE delta:cvd:{instrument} // cumulative delta, updates every 100ms
```

Payload Schema — Live Delta

Field	Type	Description
ts	int64	Nanosecond timestamp
instrument	string	Instrument symbol
bar_delta	int32	Current bar delta (resets on bar open)
cumulative_delta	int64	Session CVD
bar_volume	int64	Volume traded so far in current bar
delta_pct	float32	bar_delta / bar_volume as percentage
max_delta_session	int32	Maximum delta in any bar this session
min_delta_session	int32	Minimum delta in any bar this session

3.3 Imbalance Signals

REST — Historical Imbalances

```
GET /v1/imbalances/{instrument}/{bar_type}/{bar_size}
Parameters: from_ts, to_ts,
types=single,stacked,stacked_min=3,2nd_slot,inverse,reload
```

WebSocket Topic

```
SUBSCRIBE imbalances:{instrument} // fires on every bar close with detected
imbalances
```

Payload Schema — Imbalance Event

Field	Type	Values / Description
bar_id	string	Parent bar identifier
imbalance_type	enum	SINGLE_BUY / SINGLE_SELL / STACKED_BUY / STACKED_SELL / 2ND_SLOT_BUY / 2ND_SLOT_SELL / INVERSE_BUY / INVERSE_SELL / RELOAD_BUY / RELOAD_SELL
direction	enum	BUY or SELL
price_start	float64	Lowest price in the imbalance zone
price_end	float64	Highest price in the imbalance zone
count	int	Number of consecutive imbalances (for STACKED)
ratio	float32	Actual ratio at the strongest level (e.g. 4.7 = 470%)
zone_strength	int	1–10 score based on ratio and count
bid_vol_at_level	int32	Bid volume at the imbalance price
ask_vol_at_level	int32	Ask volume at the imbalance price
is_new_zone	bool	true if this is a new zone not seen before; false if extending
zone_id	string	Persistent zone ID — same zone keeps same ID across bars

***i** Stacked imbalance zones are persistent. The zone_id remains constant as the zone builds across multiple bars. Developers can track zone_id to monitor zone evolution over time.*

3.4 Volume Profile

REST — Historical Profile

```
GET /v1/profile/{instrument}/{profile_type}
Parameters: session_date, from_date, to_date (composite), anchor_ts (flexible)
```

WebSocket Topics

```
SUBSCRIBE profile:live:{instrument}:{profile_type} // updates every bar close
SUBSCRIBE profile:session:{instrument} // current session live profile
```

Payload Schema — Volume Profile

Field	Type	Description
profile_id	string	Unique profile identifier
instrument	string	Instrument symbol
profile_type	enum	SESSION / DAILY / WEEKLY / MONTHLY / YEARLY / FLEXIBLE / COMPOSITE
period_start / period_end	int64	Nanosecond timestamps
poc	float64	Point of Control price
vah	float64	Value Area High price
val	float64	Value Area Low price
value_area_pct	float32	Actual percentage of volume in value area (default 70%)
total_volume	int64	Total volume traded in profile period
shape	enum	D_PROFILE / P_PROFILE / B_PROFILE / THIN_PROFILE / UNCLASSIFIED
shape_confidence	float32	0.0–1.0 confidence score for shape classification
hvns	array	Array of HVN objects: { price, volume, strength_score }
lvns	array	Array of LVN objects: { price_low, price_high, gap_size }
distribution	array	Full price → volume distribution: { price, bid_vol, ask_vol, total_vol }
previous_poc	float64	POC from the previous equivalent period (carry-forward reference)
poc_migration	array	Array of { ts, poc_price } showing POC movement during session build

3.5 VWAP

REST — Historical VWAP

```
GET /v1/vwap/{instrument}/{vwap_type}
```

Parameters: session_date, anchor_ts (for anchored types), include_bands=true

WebSocket Topic

```
SUBSCRIBE vwap:{instrument} // all active VWAP levels, updates every 5 sec
```

Payload Schema — VWAP Snapshot

Field	Type	Description
ts	int64	Snapshot timestamp
instrument	string	Instrument symbol
daily_vwap	float64	Daily VWAP (anchored at session open)
weekly_vwap	float64	Weekly VWAP (anchored at Monday open)
yearly_vwap	float64	Yearly VWAP (anchored at Jan 1)
anchored_vwaps	array	Array of active anchored VWAPs: { anchor_ts, anchor_type, vwap_value, sd1_high, sd1_low, sd2_high, sd2_low }
daily_sd1_high / low	float64	Daily VWAP +/- 1 standard deviation
daily_sd2_high / low	float64	Daily VWAP +/- 2 standard deviations
weekly_sd1_high / low	float64	Weekly VWAP +/- 1 standard deviation
current_price_vs_vwap	enum	ABOVE_DAILY / BELOW_DAILY / AT_DAILY (within 0.5 ticks)
vwap_reaction_signal	enum	LONG_SETUP / SHORT_SETUP / NO_SIGNAL — based on Dale's strategy rules

3.6 orderflows.com Indicators — Dedicated Signal Streams

3.6.1 Orderflows Pulse

```
GET /v1/signals/pulse/{instrument} // historical Pulse signals
SUBSCRIBE pulse:{instrument} // live Pulse signals
```

Field	Type	Description
ts	int64	Signal timestamp
bar_id	string	Bar that generated the signal
signal_type	enum	MOMENTUM_JUMP / ORDER_FLOW_PATTERN / DELTA_IMBALANCE / VOLUME_CONCENTRATION / PRICE_IMBALANCE / PRICE_ACTION_REACTION / SWING_REFERENCE
direction	enum	LONG / SHORT / NEUTRAL
composite_score	float32	0–100 composite score across all 7 variables (>=70 = high conviction)
variable_scores	object	{ order_flow: 0-100, delta: 0-100, poc: 0-100, imbalance: 0-100, volume: 0-100, price_action: 0-100, swing: 0-100 }
timeframe_primary	string	Chart timeframe that generated the signal (e.g. 5min)

timeframe_higher	string	Higher TF used for multi-TF confirmation (e.g. 30min) — null if not enabled
multi_tf_confirmed	bool	true if signal confirmed on higher timeframe as well
suggested_entry	float64	Price level for entry (based on current order flow)
suggested_stop	float64	Suggested stop loss level
signal_id	string	Unique signal ID for tracking and backtesting

i *composite_score* >= 70 should be treated as high-conviction. All 7 *variable_scores* are exposed individually so developers can build their own weighting models.

3.6.2 Orderflows Turns

```
GET /v1/signals/turns/{instrument} // historical Turns signals
SUBSCRIBE turns:{instrument} // live Turns turning point signals
```

Field	Type	Description
ts	int64	Signal timestamp
bar_id	string	Bar that generated the turning point signal
turn_direction	enum	BULLISH_TURN / BEARISH_TURN
turn_strength	int	1–5 (5 = strongest turning point — all 6 variables aligned)
variables_aligned	int	Number of 6 variables confirming the turn (3+ = valid signal)
poc_context	enum	TURN_AT_POC / TURN_AT_HVN / TURN_AT_VAH / TURN_AT_VAL / TURN_IN_MIDDLE
delta_confirmed	bool	Delta reversal confirms the turn direction
swing_context	enum	AT_SWING_HIGH / AT_SWING_LOW / MID_RANGE
price_level	float64	Price at which the turn was detected
suggested_entry	float64	Suggested entry price after confirmation
suggested_stop	float64	Stop placement level

3.6.3 Orderflows Ratio — Top Heavy / Bottom Heavy

```
GET /v1/signals/ratio/{instrument} // historical Ratio signals
SUBSCRIBE ratio:{instrument} // live big buyer/seller signals
```

Field	Type	Description
-------	------	-------------

ts	int64	Signal timestamp
bar_id	string	Bar that generated the ratio signal
signal_type	enum	BOTTOM_HEAVY_BIG_BUYER / TOP_HEAVY_BIG_SELLER
price_level	float64	Price of the bar high (TOP_HEAVY) or bar low (BOTTOM_HEAVY)
bid_vol	int32	Bid volume at the extreme price level
ask_vol	int32	Ask volume at the extreme price level
ratio	float32	Dominant side volume / opposing volume (e.g. 8.5 = 850%)
institutional_size	bool	true if volume exceeds institutional_size_threshold config
acts_as	enum	PRICE_FLOOR / PRICE_CEILING
suggested_entry	float64	Entry on retest of detected level
suggested_stop	float64	Stop beyond the detected high/low

3.6.4 Single Prints — Last Buyer / Last Seller

```
GET /v1/signals/single_prints/{instrument}
```

```
SUBSCRIBE single_prints:{instrument} // fires when last buyer/seller detected
```

Field	Type	Description
ts	int64	Signal timestamp
bar_id	string	Bar containing the single print
signal_type	enum	LAST_BUYER_AT_HIGH / LAST_SELLER_AT_LOW
price_level	float64	Price of the single print (bar high or bar low)
volume_at_level	int32	Number of contracts at the single print price (typically 1–3)
bar_range	float64	Full bar range in ticks — provides scale context
exhaustion_score	float32	0–100 exhaustion probability score
unfinished_auction	bool	true if ask_vol at high is non-zero OR bid_vol at low is non-zero
magnet_active	bool	true if price has not yet revisited this level — still acting as magnet

3.6.5 Delta Surge

```
GET /v1/signals/delta_surge/{instrument}
```

```
SUBSCRIBE delta_surge:{instrument} // fires on delta surge detection
```

Field	Type	Description
ts	int64	Signal timestamp
surge_direction	enum	BULLISH_SURGE / BEARISH_SURGE
surge_magnitude	float32	Delta surge value relative to recent average (e.g. 3.2 = 320% of avg)
bars_building	int	Number of consecutive bars with rising delta in same direction
cvd_at_surge	int64	CVD value at time of surge
volume_at_surge	int64	Volume at time of surge
works_on_bar_chart	bool	Always true — this signal is valid on standard bar charts too

3.6.6 All Trader 7 Signals (30+ Signals)

```
GET /v1/signals/trader7/{instrument} // all Trader 7 signals
SUBSCRIBE trader7:{instrument} // all Trader 7 signals live
SUBSCRIBE trader7:{instrument}?filter=delta_divergence,poc_slingshot,market_sweep
```

The trader7 stream exposes all 30+ automatable signals. Developers can filter by signal name:

Signal Name (filter key)	Direction Type	Brief Description
accumulation_distribution	BULLISH/BEARISH	Multi-bar accumulation at lows or distribution at highs
open_poc	INFO	Current session POC as it builds in real time
aligned_pocs	BULLISH/BEARISH	Two or more session POCs at same price — very strong level
orderflows_gaps	INFO	Volume gaps inside bars — future price magnets
imbalance_reload	BULLISH/BEARISH	Second imbalance at same price in subsequent bar
volume_decline	WARNING	Sequential volume decrease — trend losing fuel
poc_wave	INFO	POC migration direction during session — shows institutional intent
delta_tail	BULLISH/BEARISH	Delta breaks outside recent range at key level
resting_liquidity	INFO	Significant volume at price without moving price — hidden S/R
vertical_liquidity	INFO	Volume stacking at consecutive levels — institution working big order

delta_breakout	BULLISH/BEARISH	Delta surges through a threshold at breakout point
retail_suck	REVERSAL	Price briefly spikes through key level to run stops then reverses
price_action_divergence	REVERSAL	Buyers/sellers gave up — Valtos U-Turn signal
prominent_poc	INFO	POC significantly above surrounding volume — strongest institutional level
orderflows_tails	BULLISH/BEARISH	Extreme price rejection tail with order flow confirmation
delta_divergence	REVERSAL	Negative delta at new high or positive delta at new low
exhaustion_prints	REVERSAL	Volume spike at move extreme with delta reversal
imbalance_reversals	REVERSAL	Imbalance in opposite direction after exhaustion
market_sweep	CONTINUATION	Large aggressive orders sweep multiple price levels
market_weakness	WARNING	New high/low on declining volume and delta
orderflows_sequencing	INFO	Consecutive bars increasing/decreasing delta — trend strength
poc_slingshot	CONTINUATION	Price breaks away from POC with strong delta
zero_print	INFO	Price level with zero contracts traded inside bar
engulfing_value_area	CONTINUATION	Bar range engulfs previous bar's entire value area
inverse_imbalance	WARNING	Imbalance opposite to prevailing trend direction
stacked_imbalance	BULLISH/BEARISH	3+ consecutive imbalances same direction
thin_prints	INFO	Price levels with very low volume — fast-move zones
orderflows_ratio	BULLISH/BEARISH	Institutional-scale bid/ask ratio at price extremes
price_exhaustion	REVERSAL	Price reaches extreme but delta does not confirm
price_defense	CONTINUATION	Institutions defending a key level against opposing pressure
multiple_imbalances	BULLISH/BEARISH	Cluster of imbalances across multiple bars at same price

4. Developer Scripting API — Building Custom Tools

Beyond consuming raw signals, developers can compose signals, define custom indicators, and register automated strategies through the Scripting API. This is the layer that enables developers to create their own trading tools on top of the engine's data.

4.1 Signal Composition API

Developers define composite signals by combining multiple OFE outputs using a declarative JSON rule language called OFE-Script:

Example: Custom Confluence Signal

Define a signal that fires when: Pulse score $\geq$ 75 AND stacked imbalance BELOW current price AND price within 3 ticks of Weekly VWAP:

```
{
  "signal_name": "my_triple_confluence_long",
  "direction": "LONG",
  "conditions": [
    { "indicator": "pulse", "field": "composite_score", "op": ">=", "value": 75 },
    { "indicator": "pulse", "field": "direction", "op": "==", "value": "LONG" },
    { "indicator": "imbalances", "field": "imbalance_type", "op": "in", "value":
["STACKED_BUY"] },
    { "indicator": "imbalances", "field": "price_end", "op": "below_current_price",
"ticks": 10 },
    { "indicator": "vwap", "field": "weekly_vwap", "op": "within_ticks", "value": 3
}
  ],
  "alert": { "webhook": "https://myserver.com/signal", "email": false }
}
```

POST /v1/scripts/signals

Body: OFE-Script JSON

Response: { script_id, status: active, estimated_latency_ms: 12 }

i Composite signals fire via WebSocket on the SUBSCRIBE scripts/{script_id} topic and optionally via webhook POST to a developer-supplied URL.

4.2 OFE-Script Language Reference

OFE-Script is a lightweight JSON-based declarative language. No custom programming language — pure JSON that any developer can write without compilation.

Condition Operators

Operator	Description	Example
----------	-------------	---------

==	Equals	{ op: '==', value: 'LONG' }
!=	Not equals	{ op: '!=', value: 'NEUTRAL' }
>=	Greater than or equal	{ op: '>=', value: 75 }
<=	Less than or equal	{ op: '<=', value: 30 }
>	Greater than	{ op: '>', value: 3 }
<	Less than	{ op: '<', value: 0 }
in	Value in list	{ op: 'in', value: ['STACKED_BUY', 'STACKED_SELL'] }
within_ticks	Price within N ticks of a reference	{ op: 'within_ticks', value: 5 }
above_current_price	Level is above current bid	{ op: 'above_current_price', ticks: 0 }
below_current_price	Level is below current ask	{ op: 'below_current_price', ticks: 0 }
crossed_above	Value crossed above threshold on this bar	{ op: 'crossed_above', value: 0 }
crossed_below	Value crossed below threshold on this bar	{ op: 'crossed_below', value: 0 }
rising_for	Value has been rising for N consecutive bars	{ op: 'rising_for', bars: 3 }
falling_for	Value has been falling for N consecutive bars	{ op: 'falling_for', bars: 3 }
zone_active	A zone (imbalance/VP level) is currently active at price	{ op: 'zone_active', zone_type: 'STACKED_BUY' }

Logical Combinators

Combinator	Description	Default
AND	All conditions must be true	Default when conditions is an array
OR	Any condition must be true	Use { or: [...] } block
NOT	Condition must be false	Use { not: { ... } } block
SEQUENCE	Conditions must occur in order within N bars	Use { sequence: [...], within_bars: 5 }
REPEAT	Condition must occur N times within M bars	Use { repeat: {...}, count: 3, within_bars: 10 }

4.3 Backtesting API

Developers run their custom scripts against historical data to validate performance before deploying live:

```
POST /v1/backtest
{
  "script_id": "my_triple_confluence_long",
  "instrument": "ES",
  "from_date": "2025-01-01",
  "to_date": "2025-12-31",
  "bar_type": "TIME",
  "bar_size": "5min",
  "slippage_ticks": 1,
  "commission_per_side": 2.25
}
```

Backtest Response Schema

Field	Type	Description
backtest_id	string	Unique backtest run identifier
status	enum	PENDING / RUNNING / COMPLETE / FAILED
total_signals	int	Total number of times the composite signal fired
winning_signals	int	Signals that resulted in price moving in signal direction
losing_signals	int	Signals that stopped out
win_rate	float32	winning_signals / total_signals as percentage
avg_win_ticks	float32	Average tick profit on winning trades
avg_loss_ticks	float32	Average tick loss on losing trades
expectancy	float32	$(win_rate \times avg_win) - (loss_rate \times avg_loss)$ in ticks
profit_factor	float32	Gross wins / Gross losses
max_drawdown_ticks	float32	Largest losing streak peak-to-trough
sharpe_ratio	float32	Risk-adjusted return metric
signals_detail	array	Per-signal detail: { ts, direction, entry_price, exit_price, result_ticks, indicators_at_signal }

***i** Backtests are processed asynchronously. Poll GET /v1/backtest/{backtest_id} for status, or subscribe to WS topic backtest:{backtest_id} for real-time progress updates.*

4.4 Alert & Webhook System

Developers configure alerts to receive notifications when their custom scripts fire:

Alert Type	Setup	Payload
WebSocket	SUBSCRIBE scripts/{script_id}	Full signal JSON payload pushed to WS connection
Webhook POST	Register URL in script definition	HTTP POST with full signal JSON to developer's server within < 500ms
Email	Register email in developer portal	Plain text + JSON attachment per signal (rate-limited to 100/day)
Telegram Bot	Bot token in developer portal	Message with signal summary to developer's Telegram chat
Discord Webhook	Discord webhook URL in portal	Formatted embed card with signal details
SMS	Phone number in developer portal	Short signal summary text (Premium plan only)

 *Webhooks must respond with HTTP 200 within 5 seconds. Failed webhooks are retried 3 times with exponential backoff. Persistent failures disable the webhook and notify via email.*

4.5 Native Plugin SDK — C++ / C# / Python

Advanced developers on the Professional or Enterprise tier can write native plugin indicators that run INSIDE the engine's data pipeline — getting access to every tick before it hits the REST API:

C++ Plugin Interface

```
// Developer implements this interface in a shared library (.dll/.so)
class IOrderFlowPlugin {
public:
    virtual void OnInit(IEngineContext* ctx, const char* config_json) = 0;
    virtual void OnTick(const TickRecord& tick) = 0;
    virtual void OnBarClose(const BarRecord& bar) = 0;
    virtual void OnSignal(SignalType type, const char* payload_json) = 0;
    virtual void OnProfileUpdate(const ProfileSnapshot& profile) = 0;
    virtual void OnVwapUpdate(const VwapSnapshot& vwap) = 0;
    virtual const char* GetOutput(const char* key) = 0;
    virtual void OnDestroy() = 0;
};
```

Python Plugin Interface

```
from ofe_sdk import OFEPlugin, TickRecord, BarRecord

class MyCustomIndicator(OFEPlugin):
    def on_init(self, ctx, config: dict):
```

```

        self.threshold = config.get('my_threshold', 75)

    def on_bar_close(self, bar: BarRecord):
        if abs(bar.bar_delta) > self.threshold:
            self.emit_signal('MY_DELTA_SPIKE', direction='LONG' if bar.bar_delta >
0 else 'SHORT')

    def on_signal(self, signal_type: str, payload: dict):
        if signal_type == 'STACKED_BUY':
            # React to engine's stacked imbalance signal
            self.log(f'Stacked buy at {payload["price_start"]}')

```

Plugin Output Exposure

- Any signal emitted by a plugin via `emit_signal()` is immediately available on the WebSocket stream under `SUBSCRIBE plugins/{plugin_id}/{instrument}`
- Plugin outputs can also be queried historically via `GET /v1/plugins/{plugin_id}/signals/{instrument}`
- Plugins can CONSUME other indicators via `ctx.get_indicator('pulse', instrument)` and compose them
- Plugins can PUBLISH custom metrics that other developers can subscribe to — enabling a marketplace

i *Plugin SDK is available for Professional tier and above. Plugins run in a sandboxed subprocess — they cannot crash the main engine. Memory and CPU usage are monitored and capped per plugin.*

5. Developer Portal — portal.orderflow-engine.com

The developer portal is the web interface for managing API keys, scripts, backtests, webhooks, and monitoring usage. It must be built as a React SPA with the following pages:

Portal Page	Features
Dashboard	Live usage stats (API calls, WS connections, signal count), subscription tier, billing next date
API Keys	Create/revoke API key pairs, set scopes (indicator groups), set IP restrictions, view last used timestamp
Scripts	Create/edit/delete OFE-Script composite signals, view signal history, enable/disable signals
Backtester	Run backtests, view results (equity curve, trade list, statistics), compare multiple scripts
Alerts	Configure webhooks, Telegram/Discord/SMS alerts, test alert delivery, view delivery log
Plugins	Upload plugin .dll/.so/.py files, configure parameters, view plugin output logs
Documentation	Interactive API docs (OpenAPI 3.1), WebSocket topic browser, OFE-Script language reference
SDK Downloads	Download client libraries (Python, C++, C#, JS, Rust), view changelog
Playground	Live interactive API explorer — test endpoints with real data without writing code
Support	Ticket system, changelog, community forum link

6. Rate Limits, Quotas & Fair Use

Limit	Basic	Standard	Professional	Enterprise
REST requests / minute	100	1,000	10,000	Unlimited
WebSocket connections	0	5	25	100
WebSocket subscriptions per connection	—	20	100	Unlimited
Historical data lookback	30 days	1 year	5 years	Full history
Active OFE-Scripts	0	5	50	Unlimited
Backtests / day	0	3	20	Unlimited
Backtest historical range	—	6 months	5 years	Full history
Plugin SDK access	No	No	Yes	Yes
Webhook alerts / day	0	100	1,000	Unlimited
Instruments covered	5	20	All	All
Data freshness (REST)	EOD	1 min lag	Real-time	Real-time + priority
Support	Docs only	Email 5 day SLA	Email 1 day SLA	Dedicated account manager

 Exceeding rate limits returns HTTP 429. The Retry-After header indicates when the quota resets. Persistent abuse results in temporary API key suspension.

7. Security — External API Layer

The external API exposes computed outputs ONLY. The C++ engine and all source code are fully isolated behind the API gateway. This section defines the complete security model for external access.

Security Control	Implementation	Purpose
API Gateway	Cloudflare or AWS API Gateway in front of all endpoints	DDoS protection, WAF, geographic filtering, TLS termination
Authentication	API Key + JWT (RS256, 15-min expiry)	Ensures only paying subscribers access data
Scoped permissions	Keys limited to specific indicator groups	Prevents over-privilege; Basic key cannot call Pulse endpoints
IP allowlisting	Optional per API key	Prevents key theft being useful without specific IP
Read-only enforcement	All external endpoints are GET or WS subscribe only	Developers cannot write to or modify engine state
Rate limiting	Per API key, per IP, per endpoint	Prevents abuse and ensures fair use across all subscribers
TLS 1.3 only	All REST and WebSocket connections require TLS 1.3	Prevents eavesdropping on signal data
Audit log	Every API call logged: timestamp, API key, endpoint, IP, response code	Compliance, abuse detection, support
Payload signing	Webhook deliveries include HMAC-SHA256 signature header	Developers verify webhooks are genuinely from OFE
Plugin sandboxing	Native plugins run in isolated subprocess with cgroup limits	Malicious or buggy plugins cannot crash the main engine
No raw tick exposure	Raw tick stream is Enterprise-only, requires additional agreement	Prevents reselling of raw market data
Key rotation	API keys can be rotated with zero downtime via portal	Limits exposure window if key is compromised

Webhook Signature Verification (Example — Python)

```
import hmac, hashlib

def verify_ofe_webhook(payload_bytes: bytes, signature_header: str, secret: str) -> bool:
    expected = hmac.new(secret.encode(), payload_bytes, hashlib.sha256).hexdigest()
    received = signature_header.replace('sha256=', '')
    return hmac.compare_digest(expected, received)
```

8. Sample Developer Workflows

The following examples show how a developer uses the SDK to build common trading tools:

8.1 Python — Live Delta + Stacked Imbalance Alert Bot

```
from ofe_sdk import OFEClient

client = OFEClient(api_key='ofe_pk_live_xxxx', secret='ofe_sk_live_xxxx')

@client.subscribe('delta:live:ES', 'imbalances:ES')
def on_data(msg):
    if msg.topic == 'delta:live:ES':
        if msg.cumulative_delta < -500: # CVD falling hard
            print(f'Bearish CVD: {msg.cumulative_delta}')

    if msg.topic == 'imbalances:ES':
        if msg.imbalance_type == 'STACKED_SELL' and msg.count >= 4:
            alert(f'Strong stacked sell: {msg.price_start}-{msg.price_end}')

client.run()
```

8.2 C# — NinjaTrader Custom Strategy Using OFE Signals

```
using OFE.SDK;

public class MyOFEStrategy : Strategy {
    private OFEWebSocket _ofe;

    protected override void OnStateChange() {
        if (State == State.Configure)
            _ofe = new OFEWebSocket('ofe_pk_live_xxxx', 'ES');
    }

    protected override void OnBarUpdate() {
        var pulse = _ofe.GetLatest<PulseSignal>();
        var vwap = _ofe.GetLatest<VwapSnapshot>();

        if (pulse?.CompositeScore >= 75 && pulse.Direction == 'LONG'
            && Close[0] > vwap?.WeeklyVwap) {
            EnterLong(1, 'OFE_Long_Entry');
        }
    }
}
```

8.3 OFE-Script — Declare a No-Code Signal in the Portal

```
{
  "signal_name": "vwap_pulse_confirmed_short",
  "instrument": "NQ",
  "bar_type": "TIME",
  "bar_size": "5min",
  "direction": "SHORT",
  "conditions": [
    { "indicator": "pulse",      "field": "direction",      "op": "==",
      "value": "SHORT" },
    { "indicator": "pulse",      "field": "composite_score", "op": ">=",
      "value": 70 },
    { "indicator": "vwap",      "field": "current_price_vs_vwap", "op": "==",
      "value": "BELOW_DAILY" },
    { "indicator": "delta",      "field": "bar_delta",      "op": "<",
      "value": 0 },
    { "indicator": "imbalances", "field": "imbalance_type",  "op": "in",
      "value": ["STACKED_SELL"] }
  ],
  "alert": {
    "webhook": "https://mybot.com/ofe-signal",
    "telegram_chat_id": "-1001234567890"
  }
}
```

8.4 Backtest Then Deploy — Full Developer Lifecycle

- Step 1: Write OFE-Script and save via portal or POST /v1/scripts/signals
- Step 2: Run backtest: POST /v1/backtest with 12-month historical range
- Step 3: Review results — win rate, expectancy, max drawdown, equity curve
- Step 4: Iterate — adjust thresholds, add/remove conditions
- Step 5: Activate signal live: PATCH /v1/scripts/signals/{script_id} { status: active }
- Step 6: Connect strategy via WebSocket or webhook to receive live alerts
- Step 7: Monitor in portal dashboard — signal count, hit rate vs backtest

9. SDK Implementation Plan

Phase	Component	Timeline	Dependencies
SDK-1	REST API — all indicator historical endpoints	Week 1–3 of API phase	Core engine must be complete
SDK-2	WebSocket server — all live indicator streams	Week 2–4 of API phase	Core engine real-time outputs
SDK-3	Authentication — API keys, JWT, scopes	Week 1–2 of API phase	PostgreSQL, Redis for token cache
SDK-4	Rate limiting + audit log	Week 2–3 of API phase	Redis for rate counters
SDK-5	OFE-Script engine — condition evaluation	Week 4–6 of API phase	All indicator streams live
SDK-6	Backtesting engine	Week 5–7 of API phase	Full historical data + OFE-Script engine
SDK-7	Webhook + alert delivery system	Week 4–5 of API phase	OFE-Script engine + queue (RabbitMQ/SQS)
SDK-8	Developer portal (React SPA)	Week 6–10 of API phase	REST API + auth complete
SDK-9	Python SDK client library	Week 3–4 of API phase	REST + WebSocket API stable
SDK-10	C# SDK client library	Week 4–5 of API phase	REST + WebSocket API stable
SDK-11	C++ native plugin framework	Week 6–8 of API phase	Core engine plugin ABI finalized
SDK-12	Documentation site (OpenAPI 3.1)	Week 7–9 of API phase	All endpoints stable
SDK-13	Interactive playground	Week 9–10 of API phase	All endpoints + portal complete
SDK-14	Pine Script bridge (TradingView)	Month 5–6	WebSocket API stable; TV webhook

Recommended Stack	Component	Technology
API Gateway	DDoS + WAF + TLS	Cloudflare or AWS API Gateway
REST API server	Indicator endpoints	Go (high performance, simple concurrency)
WebSocket server	Real-time streaming	uWebSockets (C++) or Bun.js
OFE-Script evaluator	Signal condition engine	Rust (safe, fast, no GC pauses)
Backtest engine	Historical replay	C++ (reuses core engine code)
Auth / token	JWT issuance and	Go + Redis (token cache)

	validation	
Rate limiting	Per-key, per-IP limits	Redis sliding window
Alert delivery	Webhooks + messaging	RabbitMQ + worker pool (Go)
Developer portal	Web interface	React + TypeScript + Tailwind
Documentation	OpenAPI 3.1 interactive docs	Stoplight Elements or Redocly
Database	API keys, scripts, backtests, logs	PostgreSQL + TimescaleDB (for time-series signals)
Monitoring	API health, latency, error rates	Prometheus + Grafana

END OF DOCUMENT — External Developer SDK & Indicator Exposure Specification v1.0

Covers: 3-layer SDK architecture · Complete indicator schemas · OFE-Script language · Backtesting API · Webhooks · Native Plugin SDK · Developer Portal · Security · Sample code